Assignment 4:
Build & Deploy a System

Due Week 5 at the start of class.

Goal Build and deploy a multi-service system — a background worker that polls a live data source, writes to a database, and a frontend that displays updates in real time.

🖐️ A Heads Up

This assignment is more complex than previous assignments—a full system: multiple services, running on different platforms, talking to each other. Start early. Budget more time than you think you need. Use the architecture from class as your foundation.

Overview

In class you built a live NBA scoreboard together. For this assignment, you'll build your own system using the same architecture pattern:

External Data Source → Background Worker (Railway) → Database (Supabase + Realtime) → Frontend (Next.js on Vercel)

Your system should poll a live or frequently-updating data source, store the data in Supabase, and display it on a frontend that updates in real time without the user refreshing the page.

Ideas to Explore (Pick One)

🛫 **Flight Tracker** — monitor live aircraft positions worldwide. Filter by region, show a live map with moving planes. (OpenSky Network — free)

🌤️ **Weather Dashboard** — monitor weather for favorite cities. Worker polls conditions, frontend shows live data. (Open-Meteo — free, open source)

🚌 **Transit Tracker** — live bus/train positions for a city. Users pick routes, see vehicles on a map. (CTA, MTA, MBTA — most are free)

✨ **Your Own Idea** — any live data source + background worker + realtime frontend. If it polls, stores, and pushes updates, it counts.

Requirements

- Monorepo — frontend and worker in one repo (apps/web/ and apps/worker/)
- Built with Next.js + Tailwind CSS
- Background worker deployed on Railway — polls an external data source
- Data stored in Supabase — worker writes, frontend reads
- Supabase Realtime — frontend updates live without page refresh
- User authentication via Clerk or Supabase Auth
- Users can personalize what they see (favorites, preferences)
- Environment variables in .env.local and platform dashboards
- Supabase MCP server configured
- CLAUDE.md describing your architecture
- Multiple git commits showing iteration
- Deployed to Vercel (frontend) and Railway (worker)
- Live URLs work — classmates can sign up and use it

Steps

1. Choose your data source. What are you polling? How often does it update? Is the API free? Test the endpoint before you commit.

2. Plan your architecture. Write a CLAUDE.md with your full architecture: what the worker does, what tables you need, how data flows from source to screen. This is your blueprint.

3. Set up Supabase. Create your tables via Supabase MCP. Enable Realtime on the tables the worker writes to. Set up auth. Write RLS policies.

4. Build the worker. A Node.js script that polls your data source, parses the response, and upserts into Supabase. Test it locally — watch the Supabase dashboard update.

5. Build the frontend. Next.js app that reads from Supabase, subscribes to Realtime updates, and displays personalized data based on the logged-in user's preferences.

6. Deploy the system. Worker to Railway (connect GitHub, add env vars). Frontend to Vercel (add env vars). Verify everything works end-to-end.

7. Test with a classmate. Have someone sign up on your app and use it. Does auth work? Do they see personalized data? Do updates appear in real time?

Stretch Goals

Not required, for students who finish early:

- Support multiple data sources (e.g., NBA + MLB + NHL)
- Historical data view (past scores, price history, weather trends)
- Notifications or alerts (weather drops below threshold, your team is losing)
- Data visualization (charts, graphs, maps)
- Worker health monitoring (last poll time, error count on frontend)

Submission

Deliverables

1. Vercel URL — A live link to your frontend.

2. GitHub URL — Public repository with commit history.

3. Slack Video Reflection URL (2-3 min) — Record a short video using Slack and post it to your section's Slack channel (#tuesday-night or #wednesday-night). Talk about whatever you want — walk us through your system, show something you're proud of, explain what broke and how you fixed it. Informal — no slides, no script. Screen share encouraged. (Slack URLs can be found by hovering over your message, clicking on the 3 vertical dots: you should see “Copy link”).

Assessment

A complete assignment has:

- ☐ A live Vercel URL that works — classmates can sign up and see live data
- ☐ A running Railway worker that polls and updates the database
- ☐ Supabase Realtime pushing updates to the frontend
- ☐ Auth with personalized data (users see their own preferences)
- ☐ A GitHub repo with multiple commits showing iteration
- ☐ A video reflection posted to your section's Slack channel

You'll be assessed on whether you deployed a working multi-service system, understood the architecture, and built something your classmates can use.

Notes

This is the last structured assignment. Starting next week, you'll be working on your individual projects for the rest of the quarter. Use this assignment to push your understanding of systems architecture — the patterns you learn here (background workers, realtime subscriptions, multi-platform deployment) will likely show up in your projects.

Refer to the Week 4 lesson plan and slides in Google Classroom for the step-by-step build we did in class.

Explore Claude Code + Codex

CLAUDE.md — Describe your full architecture: worker, database, frontend, data flow.

AGENTS.md — Create one alongside CLAUDE.md if you want to try Codex this week.

Supabase MCP — claude mcp add --transport http supabase https://mcp.supabase.com/mcp

Codex — npm i -g @openai/codex — try building one piece with Codex and compare.

Questions? Reach out on Slack or come to office hours.